

Minimal-Intervention Target Retrieval in Cluttered Scenes with Grasp-Conditioned Dependency Graphs

Wanhao Niu^{1,*} Qiyao Ke^{1,*} Yuan Sun² Chongrui Zhang³
Huaidong Zhou^{1,†} Yongfeng Rong¹ Shuang Zhang⁴ Fuchun Sun^{1,†}

¹Department of Computer Science and Technology, Tsinghua University

²School of Reliability and Systems Engineering, Beihang University

³Department of Mechanical Engineering, Tsinghua University

⁴School of Engineering, Hebei Normal University

Abstract: Minimal-intervention target retrieval asks a robot to extract a target object from clutter while limiting non-target removals. The challenge is that a single scene may support multiple candidate grasps, and whether an object needs to be removed depends on the chosen grasp and the current execution stage, so scene-level clutter cues alone are insufficient. We address this with grasp-conditioned dependency graphs, where candidate grasps are action nodes and typed object-grasp edges represent approach- and lift-stage blockers. By tying blockers to concrete candidate grasps, the graph lets the planner distinguish objects that merely surround the target from those that are likely to block a particular grasp. We evaluate these dependencies using single-object intervention tests on 12,985 scene-target samples from 500 multi-layer dense scenes, yielding 6,766,097 labeled graph edges, typed dependencies, and oracle minimal blocker sets. An action-centric edge-context model predicts object-grasp dependencies from observed scenes, and a short-prefix planner ranks blockers, selects a target grasp, and replans after each intervention. Stored-label planning provides a large-scale controlled test of the success-removal trade-off, closed-loop MuJoCo rollouts test the same policy under physical scene updates, and hardware trials validate the execution interface.

Keywords: Cluttered Manipulation, Grasping, Relational Prediction, Robot Learning

1 Introduction

Robots that retrieve a target from a dense stacked-clutter bin or shelf often cannot simply grasp the target first [1, 2, 3, 4]. Nearby objects may block the approach path, prevent closing or suction, collide during lift, or hide better target grasps [5, 6, 7]. Removing every nearby object is a reliable way to make progress, but it is slow and disruptive because it increases execution time, changes the scene for future tasks, and can move objects that were never relevant to the target. A retrieval policy should therefore know not only that the scene is cluttered, but which interventions are actually necessary.

Existing target-retrieval systems usually answer a coarser question. Mechanical-search methods remove visible or nearby objects to expose the target [1]; X-Ray and LAX-RAY estimate hidden occupancy or shelf access to choose where clearing may help [2, 3]; learned policies map observations to closed-loop removal actions [5]; and support- or reasoning-based methods identify objects

*Equal contribution.

†Corresponding authors: Huaidong Zhou (hdzhou@tsinghua.edu.cn) and Fuchun Sun (fcsun@tsinghua.edu.cn).

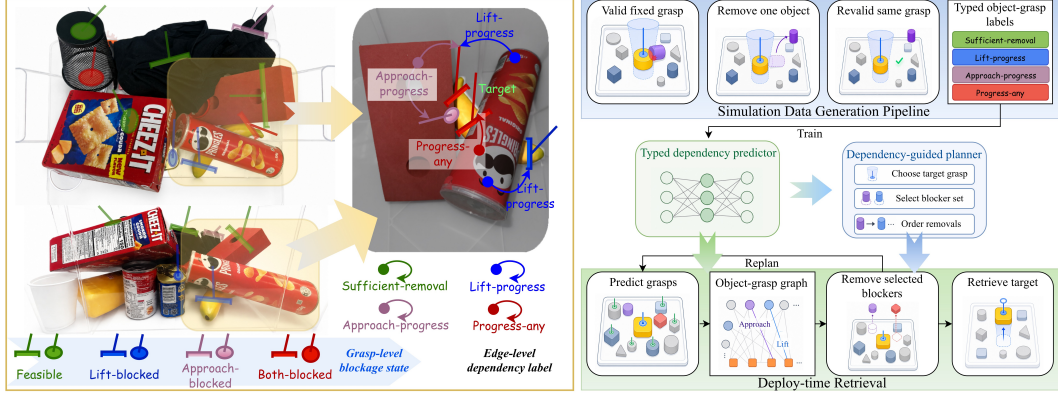


Figure 1: Method overview. Blockage in cluttered retrieval depends on the candidate grasp. Within the same scene, candidate grasps on the target object can be feasible, approach-blocked, lift-blocked, or both-blocked, and each neighboring object has a typed dependency with respect to a particular target grasp. Our pipeline generates object-grasp dependency labels using single-object removal tests, trains a typed dependency predictor, and uses the predictions in a closed-loop planner that selects a target grasp, removes predicted blockers, and replans until the target is retrieved.

that are structurally or semantically relevant to retrieval [6, 7]. These signals can prioritize plausible objects to move, but they do not directly label the causal dependency between a specific object and a specific target grasp.

This gap matters once a robot observes multiple possible grasps on the target. Distance, visibility, occupancy, and support are useful proxies in prior retrieval and clearing methods [1, 2, 3, 5, 6, 7], but none answers the action-level question of whether this object blocks this grasp during approach, closing or suction, or lift. An object close to the target need not block the chosen approach, a farther object can still obstruct a lift, and some failures come from proposal geometry or bin contact rather than any removable object. Minimal-intervention retrieval therefore requires an action-conditioned notion of dependency. For a concrete candidate grasp, the policy must estimate which observed objects block execution and at which stage.

We therefore model retrieval as grasp-conditioned dependency prediction, as summarized in Fig. 1. Candidate grasps are action nodes, non-target objects are object nodes, and object-grasp edges encode whether an object blocks a concrete grasp. Single-object interventions provide typed labels, an edge-context predictor learns these sparse dependencies from deployable observations, and a short-prefix planner ranks blockers, selects a low-blocker grasp, and replans after each removal.

Our contributions are as follows.

1. A grasp-conditioned dependency benchmark and evaluation protocol with 500 multi-layer dense scenes, 12,985 target samples, 313,228 candidate grasps, and 6,766,097 labeled object-grasp graph edges from single-object interventions.
2. An action-centric edge-context predictor and short-prefix planner that score sparse object-grasp dependencies from deployable features, choose a low-risk target grasp, and replan after each selected removal.
3. Evaluations of the same dependency representation across offline prediction, stored-label planning, closed-loop MuJoCo rollouts, and hardware trials.

2 Related Work

Datasets for cluttered manipulation and retrieval. Manipulation datasets provide supervision at different levels. General benchmarks emphasize task diversity, demonstrations, and policy learn-

ing [8, 9, 10, 11, 12, 13]. Grasp-centric datasets and methods focus on proposing or scoring stable grasps in clutter [14, 15, 16, 17, 18, 19, 20, 21, 22], while FetchBench evaluates target fetching [4]. These labels are usually attached to demonstrations, grasp feasibility, tactile stability, or final task success. Minimal retrieval instead needs local intervention supervision for each candidate target grasp, identifying which surrounding objects are necessary blockers and at which action stage they matter. Our dataset provides this signal through single-object removal tests that produce typed object-grasp dependency labels and minimal blocker sets (MBS).

Relational prediction for physical interaction. Graph networks provide an inductive bias for entity-relation reasoning, typed relations, latent interactions, and physical dynamics. Representative formulations include relation networks, relational graph convolution, neural relational inference, and graph-network simulators [23, 24, 25, 26]. In manipulation, relational models improve target-driven grasping [27], grasp stability analysis [19, 22], support reasoning for retrieval [6], and action-conditioned scene graph construction [28]. Most existing abstractions predict relations at the object-object, grasp-instance, or whole-scene level. This collapses over candidate retrieval actions, even though the same object can be irrelevant for one target grasp, block the approach of another, and obstruct the lift of a third. As a result, a planner that treats blockers as scene-level or object-level properties may remove unnecessary objects or discard feasible grasps. We instead represent candidate target grasps as explicit action nodes and predict typed dependencies on object-grasp edges.

Planning for target retrieval with minimal disturbance. Target retrieval is often treated as sequential search and manipulation. Mechanical Search removes objects until extraction is possible [1], later systems estimate hidden occupancy or shelf access [2, 3], learned policies map observations to closed-loop removals [5], and support- or language-guided methods reason about useful grasping actions [6, 7]. Minimal disturbance is also related to minimum constraint removal, task-and-motion planning, and learned action-dependency heuristics [29, 30, 31, 32]. These methods usually reason over global search state, visibility, support, or high-level dependencies, rather than comparing candidate target grasps by their necessary blockers. For a robot that already observes several candidate target grasps, the local planning problem is to choose a low-blocker grasp, remove predicted blockers, and replan after each intervention.

3 Method

We formulate retrieval as choosing a candidate target grasp or removing a blocking object and replanning until the target is retrieved. An episode contains a scene $\mathcal{S}$, target object t , observed non-target objects $\mathcal{O}_t = \{o_i\}_{i=1}^N$, and candidate target grasps $\mathcal{G}_t = \{g_j\}_{j=1}^M$.

The method has two parts, intervention-derived supervision and dependency-guided prediction and planning. The learned predictor and planner are used at deployment time; the intervention oracle is used only for supervision and evaluation.

3.1 Intervention-Derived Supervision

The supervision procedure fixes a candidate target grasp and asks how the scene changes when one surrounding object is removed. The action interface accepts parallel-jaw proposals from Any-Grasp [18] and suction proposals in the SuctionNet format [16]. The dense benchmark retains both parallel-jaw and suction candidates; proposal filtering details are in the appendix.

Let $V(\mathcal{S}, t, g_j)$ be a modality-aware staged validator. It returns a feasibility bit $f(r)$, terminal stage $c(r)$, and a numeric progress rank $\rho(r)$; the exact stage-to-rank mapping is given in the appendix. Removing an object may not make a grasp fully feasible, but it can still move the terminal failure to a later stage.

We evaluate each grasp in the full scene and then after each single-object removal to ask whether object o_i is a blocker for grasp g_j , not whether it is merely close to the target. Already feasible grasps

receive zero dependency labels, and failures caused by bin walls, support surfaces, invalid approach directions, or invalid contact/seal geometry are labeled as non-recoverable by object removal.

Each edge receives four binary labels, sufficient-removal, approach progress, lift progress, and their progress-any union; the all-zero case is the no-dependency state. The sufficient-removal label is deliberately stronger than the typed progress labels because it is positive only when removing o_i alone makes the fixed grasp g_j feasible. Approach and lift labels are weaker stage-progress signals and can be positive even when several blockers must be removed before the grasp succeeds. This multi-label vector is used for training and auditing, while grasp-level blockage states in figures are obtained by aggregating typed approach and lift dependencies over the remaining objects.

The same intervention oracle enumerates bounded minimal blocker sets (MBS) up to an oracle search budget $K_{\text{oracle}} = 3$, the maximum number of non-target objects considered in a successful removal subset. These sets provide set-level evaluation and headroom estimates; the predictor is trained on the single-object edge labels. A jointly blocking set can remain at the same failure stage after any one member is removed, so a member need not have a positive single-object label. Separating approach and lift stages matters because the same object can block different parts of the execution chain for different grasps. An approach collision usually means the blocker sits in the motion path, whereas a lift-stage failure can reflect contact or support geometry that only becomes solvable after a different removal. The labels therefore preserve more structure than a single feasibility bit.

3.2 Dependency Prediction and Planning

Each scene-target instance is represented as an action-centric bipartite graph whose nodes are observed non-target objects and candidate target grasps. We connect every object to every candidate grasp, so each edge is one possible dependency. The target itself is represented through target-relative features and through the shared identity of the grasp nodes. This structure keeps the prediction unit aligned with the label unit because an edge says whether object o_i blocks grasp g_j .

Object features include detector output, visibility, depth, size, and target-relative position; grasp features include modality, score, pose, and approach/lift geometry. Edge features encode relative geometry and motion alignment, and all features are available from perception and grasp proposals.

Each episode specifies the target identity. Its detected 2D box or instance mask defines the target region: simulation supplies RGB-D and instance masks, while the hardware system uses YOLO boxes and registered depth. The target region selects points for target-grasp generation, and a proposal is assigned to the target when its projected center lies inside that region. The predictor consumes the resulting structured graph, rather than raw images or point clouds.

The object, grasp, and object-grasp edge vectors contain 36, 28, and 22 channels, respectively. Box and depth statistics provide estimated object centers and sizes; grasp features provide SE(3) pose, score, motion directions, and jaw or suction geometry. Path clearance is the estimated object-to-approach/lift-segment distance minus object radius. Motion alignment is the dot product between the normalized object-to-grasp vector and the approach or lift direction. The four output probabilities below express stage progress or successful recovery for each fixed object-grasp pair. Appendix A2.1 lists the input channels and their construction.

For object o_i and grasp g_j , let $\mathbf{x}_i^o$, $\mathbf{x}_j^g$, and $\mathbf{e}_{ij}$ denote object, grasp, and object-grasp edge features. Separate endpoint encoders and an edge encoder produce the initial dependency-edge state $\mathbf{u}_{ij}^0$. At each layer, the model aggregates a per-grasp context over candidate blockers for the same action and a per-object context over the alternative grasps affected by the same object. That comparative context is the key signal. This context lets each edge be scored relative to competing blockers for the same grasp and relative to other grasps affected by the same object. Residual context blocks update the edge state while preserving the edge as the prediction unit; full equations are in the appendix. The final decoder uses a shared trunk with label-specific heads and predicts

$$\hat{\mathbf{p}}_{ij} = [\hat{p}_{ij}^{\text{prog}}, \hat{p}_{ij}^{\text{succ}}, \hat{p}_{ij}^{\text{app}}, \hat{p}_{ij}^{\text{lift}}].$$

Training uses weighted binary cross entropy over the four sparse labels and a small consistency penalty between $\hat{p}_{ij}^{\text{prog}}$ and the maximum of the approach- and lift-progress typed heads. The predictor is selected by validation AP on y^{prog} , and all four heads are retained for auditing and planning. Detailed [label definitions, weighting, and objective terms](#) are provided in the appendix.

The planner is intentionally lightweight. It does not solve a full rearrangement problem, but scores short blocker prefixes and uses them to choose the next intervention. For each object o_i and grasp g_j , we form a scalar dependency strength D_{ij} . The main planner uses the sufficient-removal head $\hat{p}_{ij}^{\text{succ}}$, while the other heads are kept for ablations and auditing. Progress-any supplies denser supervision for model selection because it also labels partial stage advances; sufficient-removal aligns the planning score with complete recovery of a fixed grasp. For each grasp g_j , objects are ranked by D_{ij} , and $\mathbf{q}_j^k$ denotes the first k removals with $0 \leq k \leq K$, where K is the maximum number of non-target removals allowed in an episode.

Given a grasp-prefix pair, we score the residual dependency risk after removing the prefix,

$$R(g_j, k) = \sum_{o_i \in \mathcal{O}_t \setminus \mathbf{q}_j^k} D_{ij},$$

and select the pair that minimizes

$$J(g_j, k) = \lambda_R R(g_j, k) + \lambda_K k - \alpha s_j.$$

This objective favors short prefixes for high-scoring target grasps. The grasp score s_j penalizes weak target grasps, while the removal penalty discourages additional interventions. Because recoverable targets usually have short MBSs, the planner compares short prefixes rather than long removal sequences.

At execution time, the robot either attempts the selected target grasp or removes the first object in $\mathbf{q}_j^k$ and replans on the updated scene. This receding-horizon step avoids committing to a full open-loop clearing plan. The thresholded blocker set variant is retained as an ablation.

4 Experiments

The experiments evaluate the pipeline from supervision to prediction to planning, using complementary offline, stored-label, closed-loop simulation, and ablation protocols. The stored-label protocol provides the large-scale controlled test of the success/removal trade-off; the closed-loop MuJoCo rollouts test the same policy under physical scene updates and refreshed grasp proposals; and the real-robot trials validate the hardware execution interface. AP is the main prediction metric because dependency positives are sparse. Retrieval is evaluated as a coupled objective: a useful policy should increase target success without spending the full removal budget, so planning reports both target success and non-target removals, with fuller diagnostics in [the appendix](#).

4.1 Benchmark Construction and Sparse-Label Evidence

The benchmark contains 500 multi-layer dense scenes, 12,985 target samples, 313,228 candidate target grasps, and 6,766,097 labeled object-grasp graph edges. Each scene contains 25–27 dropped-and-settled objects in a $0.22 \times 0.16 \times 0.20$ m bin. Its construction first synthesizes candidate target actions and then runs single-object intervention tests to label which object blocks which grasp. Across the observed graph edges, 1.37% are progress-positive, providing sparse-label evidence for the regime the planner must handle. The bounded oracle solves 8,949 targets within three removals: 2,119 directly and 6,830 after one to three removals. The candidate pool contains 161,304 parallel-jaw and 151,924 suction grasps; current-clutter proposals are augmented with isolated-target candidates at the target’s current pose, with no augmentation for fully hidden targets. Scenes are split into 350 for training, 75 for validation, and 75 for testing, containing 9,090, 1,950, and 1,945 targets, respectively.

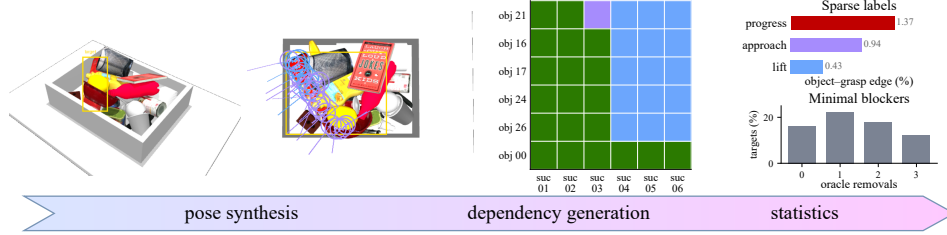


Figure 2: Benchmark construction workflow and sparse-label evidence.

Figure 2 summarizes the benchmark-building pipeline. Pose synthesis produces candidate target grasps, single-object intervention tests generate object-grasp dependency matrices, and aggregate statistics show that positive dependency labels and oracle MBS annotations are sparse. The same scene can contain many target grasps, and each object-grasp edge can carry a different dependency state.

4.2 Dependency Prediction and Minimal-Intervention Planning

Dependency prediction is evaluated before planning so that the sparse edge scores can be audited directly. Models are selected by validation AP on progress-any, and Table 1 compares a geometric score, Edge-MLP, a G2N2-style graph baseline [27], and the retained edge-context model.

Table 1: Dependency prediction.

Method	AP $\uparrow$	F1 $\uparrow$	Blocker IoU $\uparrow$
Geometry	3.3 ± 0.0	7.1 ± 0.0	4.4 ± 0.0
Edge-MLP	31.5 ± 0.5	37.3 ± 0.4	24.8 ± 0.3
G2N2 [27]	38.3 ± 0.5	42.2 ± 0.5	29.3 ± 0.3
Ours	40.0 ± 0.3	43.5 ± 1.1	30.7 ± 1.1

Edge-MLP scores each object-grasp pair independently from the same structured inputs, while G2N2-style scoring propagates graph messages. The edge-context model aggregates across objects sharing a grasp and grasps sharing an object. These comparisons test how relational context changes dependency estimation for the same candidate actions.

The edge-context model reaches 40.0 AP, 43.5 F1, and 30.7 blocker IoU, compared with 38.3/42.2/29.3 for the G2N2-style baseline and 31.5/37.3/24.8 for Edge-MLP. Geometry alone reaches 3.3 AP because proximity does not tell which object is causal for a candidate grasp. The edge-context model compares each edge against other blockers of the same grasp and against alternative grasps affected by the same object. On nontrivial oracle MBS cases, its top-3/top-5 rankings recover 54.0%/73.3% of blocker objects, compared with 48.9%/67.2% for G2N2 (Appendix A2).

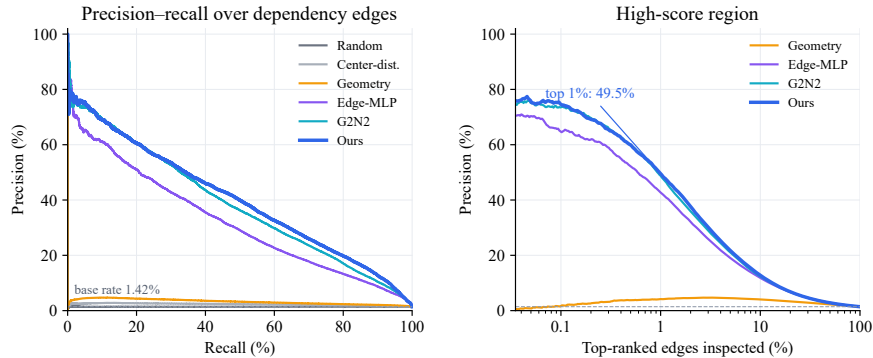


Figure 3: Dependency prediction on held-out dense scenes, with curves from model seed 7.

Figure 3 gives the visual view behind the prediction table. The left panel shows precision-recall over all dependency edges, while the right panel zooms into the top-ranked region where the planner draws short blocker prefixes.

The planner then evaluates downstream retrieval while holding the 1,945 test targets fixed. After each planned removal, the benchmark deletes the selected object and replans over the remaining objects and grasps. Table 2 compares direct grasping, nonlearned clearing baselines, prior-method adapters from Mechanical Search and X-Ray-style support reduction [1, 2, 3], aggressive dependency clearing, the proposed planner, and an Oracle MBS diagnostic reference under the benchmark search budget.

Table 2: Stored-label planning.		
Method	Success $\uparrow$	Removals $\downarrow$
Direct	8.3	0.00
Nearest	26.2	5.00
Mech. Search [1]	11.5	2.32
X-Ray-style [2, 3]	14.9	2.32
Highest dep.	40.5	5.00
Ours	31.9	2.32
Oracle MBS	67.7	0.95

Direct target grasping succeeds on 8.3% of targets. The proposed planner reaches 31.9% success with 2.32 removals, outperforming the non-aggressive clearing baselines while removing 2.68 fewer objects than nearest-to-target clearing. Highest-dependency clearing reaches 40.5% success by spending the full five-removal budget. The validation-selected operating point has utility 0.134 under the success-minus-removal objective, compared with 0.005 for highest-dependency clearing.

Budget sweeps in the appendix show the same pattern across operating points: more budget can buy success, but often by removing objects that are not needed for the selected target grasp. Figure 4 therefore visualizes the central planning trade-off: remove blockers predicted to unblock a concrete target action, rather than clear clutter indiscriminately.

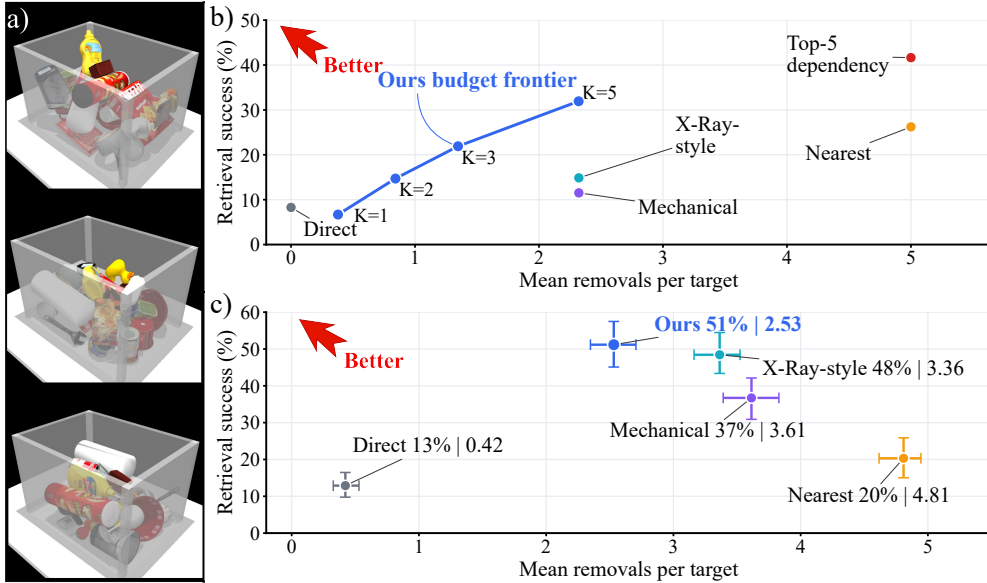


Figure 4: Dense-scene retrieval. (a) Example scenes. (b) Planning success/removal trade-off on 75 test scenes (1,945 targets), with budgets $K = 1, 2, 3, 5$. (c) Closed-loop MuJoCo results on 10 test scenes (256 targets); bars show 95% scene-cluster bootstrap intervals.

4.3 Closed-Loop MuJoCo and Real-Robot Evaluation

The stored-label benchmark is large and controlled, but it abstracts away contact-rich scene evolution after each removal. Closed-loop MuJoCo rollouts re-execute the same planning idea with physical updates, refreshed target proposals, and terminal failures caused by approach, lift, and missing proposals. Table 3 summarizes both the 256-target dense single-object rollout and an ordered-sequence

protocol over four 15-object dense scenes, with ten designated targets and four shared orders per scene. In the single-target rollout, the learned dependency planner succeeds on 51.2% of targets with 2.53 removals, compared with 48.4% and 3.36 removals for the X-Ray-style adapter. All methods share a proposal-recovery rule: when the target is fully invisible or has no current-clutter grasp proposal, the controller removes the largest visible non-target by physical proxy-box volume, resettles, and observes again. Recovery and planner interventions share the five-removal budget. The paired success difference from X-Ray-style is 2.73 percentage points (95% scene-cluster interval: -4.69 to 9.88). In the ordered protocol, Ours retrieves 137/160 targets with 183 relocations, compared with 129/160 and 281 for X-Ray. This corresponds to 8.56/10 targets per sequence and 1.34 relocations per retrieved target for Ours. The appendix gives the shared placement protocol and model-seed results. Figure 5 shows the real-robot hardware setup and three hardware examples in which target objects are retrieved after dependency-guided decluttering.

Table 3: Closed-loop retrieval.

Method	256-t succ. $\uparrow$	Rem. $\downarrow$	Ord. comp. $\uparrow$	Reloc./retr. $\downarrow$
Direct	12.9	0.42	1.81/10	1.90
Nearest	20.3	4.81	5.25/10	9.52
Mechanical [1]	36.7	3.61	5.44/10	4.98
X-Ray [2, 3]	48.4	3.36	8.06/10	2.18
Ours	51.2	2.53	8.56/10	1.34

The real-robot experiment validates the same dependency-guided closed-loop planner on hardware. Table 4 reports the mean over three repeated 20-trial runs. Direct averages 7.3 successful trials with zero removals. The Mechanical Search adapter reaches 15.0 successes with 7.3 removals, while Ours reaches 18.3 successes with 4.7 removals. More real-robot execution content is provided in the appendix (Fig. A4-E) and the accompanying video.

Table 4: Real-robot retrieval.

Method	Trials	Success	Rem.
Direct	20	7.3	0.0
Mech.	20	15.0	7.3
Ours	20	18.3	4.7

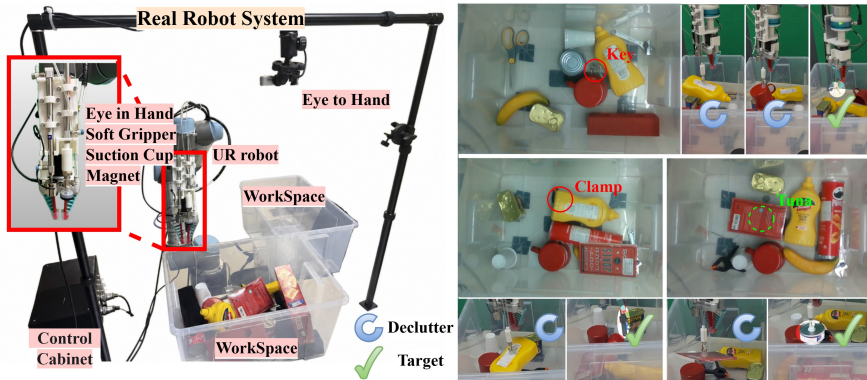


Figure 5: Real-robot system and retrieval examples. The hardware platform is shown on the left, with three real-robot examples on the right in which target objects are retrieved after dependency-guided decluttering.

4.4 Ablation Summary

The appendix contains the full ablation suite. Removing edge-context aggregation lowers dependency AP from 40.0 to 34.5 and F1 from 43.5 to 39.0. Object-only scoring shifts the planner toward higher-removal solutions because it cannot separate nearby clutter from grasp-specific blockers.

Planner sweeps report how the selected operating point changes when the trade-off is shifted toward either success or fewer removals.

On the sufficient-removal label used for planning, Ours reaches 22.06 AP, compared with 16.96 for G2N2 and 12.08 for Edge-MLP (Table A2-D). In the budgeted planner, sufficient-removal scoring yields 31.9% success with 2.32 removals, while progress-any scoring yields 27.1% with 2.30 removals (Table A3-C). This comparison links the choice of prediction head to retrieval: scores for complete grasp recovery improve success at a similar intervention cost, while stage-progress labels provide denser supervision for learning.

5 Conclusion

This work addresses target retrieval in dense stacked clutter by treating blockage as a grasp-conditioned relation rather than an object- or scene-level property. The same object may be irrelevant to one target grasp, block the approach of another, or obstruct the lift of a third. We instantiate this view as typed object-grasp dependencies, generate supervision with single-object removal tests, and train a dependency predictor that supports closed-loop planning. By selecting grasps, removing predicted blockers, and replanning after each intervention, the planner improves the success/removal trade-off across stored-label planning, closed-loop MuJoCo evaluation, and hardware validation. The resulting object-grasp graph records which objects are predicted to block each candidate grasp and at which execution stage, giving the planner an action-level explanation for each intervention.

6 Limitations

Our study decouples dependency reasoning from low-level grasp and suction proposal generation. In both labeling and evaluation, proposals supplied through the grasp and suction interfaces are treated as given candidate actions, so uncertainty in the proposal sources is not quantified by the benchmark. The intervention labels are also tied to a staged simulator-based validator and a bounded single-object removal protocol, which makes them costly to obtain and limits the explanation space to failures caused by removable clutter objects. In addition, the planner is short-horizon: when dependency scores are wrong, it may remove unnecessary objects or commit to a suboptimal target action.

Training uses fixed contact and friction settings, so different surfaces or highly concave objects may change the dependencies encountered at execution. The action set covers grasping, suction, and object removal; pushing, sliding, tilting, regrasping, and composite actions remain outside the current evaluation.

A natural next step is to make action generation and dependency reasoning part of the same closed-loop process. Instead of treating grasp and suction proposals as fixed inputs, a future system could propagate proposal uncertainty into blocker prediction, refine candidate actions after each scene update, and actively query or simulate only the interventions that are most informative for planning. This would move the framework toward longer-horizon rearrangement, where the robot reasons jointly about which action to try, which object to move, and when additional evidence is needed before disturbing the scene.

Acknowledgments

This work was supported by the Beijing Natural Science Foundation (Grant Nos. L233006 and L253006) and the Joint Funds of the National Natural Science Foundation of China (Grant No. U22A2057).

References

- [1] M. Danielczuk, A. Kurenkov, A. Balakrishna, M. Matl, D. Wang, R. Martin-Martin, A. Garg, S. Savarese, and K. Goldberg. Mechanical search: Multi-step retrieval of a target object occluded by clutter. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 1614–1621. IEEE, 2019.
- [2] M. Danielczuk, A. Angelova, V. Vanhoucke, and K. Goldberg. X-ray: Mechanical search for an occluded object by minimizing support of learned occupancy distributions. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9577–9584. IEEE, 2020.
- [3] H. Huang, M. Dominguez-Kuhne, V. Satish, M. Danielczuk, K. Sanders, J. Ichnowski, A. Lee, A. Angelova, V. Vanhoucke, and K. Goldberg. Mechanical search on shelves using lateral access X-RAY. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2045–2052. IEEE, 2021.
- [4] B. Han, M. Parakh, D. Geng, J. A. Defay, G. Luyang, and J. Deng. FetchBench: A simulation benchmark for robot fetching. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 3053–3071. PMLR, 2025. URL <https://proceedings.mlr.press/v270/han25a.html>.
- [5] A. Kurenkov, J. Taglic, R. Kulkarni, M. Dominguez-Kuhne, A. Garg, R. Martin-Martin, and S. Savarese. Visuomotor mechanical search: Learning to retrieve target objects in clutter. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8408–8414. IEEE, 2020.
- [6] Y. Li, R. Wu, H. Lu, C. Ning, Y. Shen, G. Zhan, and H. Dong. Broadcasting support relations recursively from local dynamics for object retrieval in clutters. In *Proceedings of Robotics: Science and Systems (RSS)*, Delft, Netherlands, July 2024. URL <https://www.roboticsproceedings.org/rss20/p070.html>.
- [7] Y. Qian, X. Zhu, O. Biza, S. Jiang, L. Zhao, H. Huang, Y. Qi, and R. Platt. ThinkGrasp: A vision-language system for strategic part grasping in clutter. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 3568–3586. PMLR, 2025. URL <https://proceedings.mlr.press/v270/qian25c.html>.
- [8] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison. RL Bench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 5(2):3019–3026, 2020.
- [9] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani, and J. Lee. Transporter networks: Rearranging the visual world for robotic manipulation. In *Proceedings of the 2020 Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 726–747. PMLR, 2021. URL <https://proceedings.mlr.press/v155/zeng21a.html>.
- [10] M. Shridhar, L. Manuelli, and D. Fox. CLIPort: What and where pathways for robotic manipulation. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 894–906. PMLR, 2022. URL <https://proceedings.mlr.press/v164/shridhar22a.html>.
- [11] T. Mu, Z. Ling, F. Xiang, D. Yang, X. Li, S. Tao, Z. Huang, Z. Jia, and H. Su. ManiSkill: Generalizable manipulation skill benchmark with large-scale demonstrations. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [12] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. RoboCasa: Large-scale simulation of household tasks for generalist robots. In *Proceedings of Robotics: Science and Systems (RSS)*, Delft, Netherlands, July 2024. URL <https://www.roboticsproceedings.org/rss20/p050.html>.

- [13] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. D. Fagan, J. Hejna, M. Itkina, M. Lepert, Y. J. Ma, P. T. Miller, J. Wu, S. Belkhale, S. Dass, H. Ha, A. Jain, A. Lee, Y. Lee, M. Memmel, S. Park, I. Radosavovic, K. Wang, A. Zhan, K. Black, C. Chi, K. B. Hatch, S. Lin, J. Lu, J. Mercat, A. Rehman, P. R. Sanketi, A. Sharma, C. Simpson, Q. Vuong, H. R. Walke, B. Wulfe, T. Xiao, J. H. Yang, A. Yavary, T. Z. Zhao, C. Agia, R. Baijal, M. G. Castro, D. Chen, Q. Chen, T. Chung, J. Drake, E. P. Foster, J. Gao, D. A. Herrera, M. Heo, K. Hsu, J. Hu, D. Jackson, C. Le, Y. Li, R. Lin, Z. Ma, A. Maddukuri, S. Mirchandani, D. Morton, T. Nguyen, A. O'Neill, R. Scalise, D. Seale, V. Son, S. Tian, E. Tran, A. E. Wang, Y. Wu, A. Xie, J. Yang, P. Yin, Y. Zhang, O. Bastani, G. Berseth, J. Bohg, K. Goldberg, A. Gupta, A. Gupta, D. Jayaraman, J. J. Lim, J. Malik, R. Martín-Martín, S. Ramamoorthy, D. Sadigh, S. Song, J. Wu, M. C. Yip, Y. Zhu, T. Kollar, S. Levine, and C. Finn. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems (RSS)*, Delft, Netherlands, July 2024. URL <https://www.roboticsproceedings.org/rss20/p120.html>.
- [14] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. Aparicio Ojea, and K. Goldberg. Dex-Net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics. In *Proceedings of Robotics: Science and Systems (RSS)*, Cambridge, Massachusetts, July 2017. URL <https://www.roboticsproceedings.org/rss13/p58.html>.
- [15] H.-S. Fang, C. Wang, M. Gou, and C. Lu. GraspNet-1billion: A large-scale benchmark for general object grasping. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11444–11453, 2020.
- [16] H. Cao, H.-S. Fang, W. Liu, and C. Lu. SuctionNet-1billion: A large-scale benchmark for suction grasping. *IEEE Robotics and Automation Letters*, 6(4):8718–8725, 2021.
- [17] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox. Contact-GraspNet: Efficient 6-DoF grasp generation in cluttered scenes. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 13438–13444. IEEE, 2021.
- [18] H.-S. Fang, C. Wang, H. Fang, M. Gou, J. Liu, H. Yan, W. Liu, Y. Xie, and C. Lu. AnyGrasp: Robust and efficient grasp perception in spatial and temporal domains. *IEEE Transactions on Robotics*, 39(5):3929–3945, 2023.
- [19] W. Niu, Z. Zhu, H. Wang, and C. Zhuang. Customizable 6 degrees of freedom grasping dataset and an interactive training method for graph convolutional network. *Engineering Applications of Artificial Intelligence*, 138:109320, 2024.
- [20] J. Li and D. J. Cappelleri. Sim-grasp: Learning 6-DOF grasp policies for cluttered environments using a synthetic benchmark. *IEEE Robotics and Automation Letters*, 9(9):7645–7652, 2024.
- [21] B. Hu, X. Zhu, D. Wang, Z. Dong, H. Huang, C. Wang, R. Walters, and R. Platt. OrbitGrasp: SE(3)-equivariant grasp learning. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2456–2474. PMLR, 2025. URL <https://proceedings.mlr.press/v270/hu25b.html>.
- [22] W. Niu, Z. Zhu, J. Zheng, and C. Zhuang. Visual-tactile grasp dataset and grasp margin matrix analysis for stability evaluation. *IEEE Transactions on Robotics*, 42:1515–1534, 2026.
- [23] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, C. Gulcehre, F. Song, A. Ballard, J. Gilmer, G. Dahl, A. Vaswani, K. Allen, C. Nash, V. Langston, C. Dyer, N. Heess, D. Wierstra, P. Kohli, M. Botvinick, O. Vinyals, Y. Li, and R. Pascanu. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.

- [24] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web: 15th International Conference, ESWC 2018*, volume 10843 of *Lecture Notes in Computer Science*, pages 593–607. Springer, 2018.
- [25] T. Kipf, E. Fetaya, K.-C. Wang, M. Welling, and R. Zemel. Neural relational inference for interacting systems. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 2688–2697. PMLR, 2018. URL <https://proceedings.mlr.press/v80/kipf18a.html>.
- [26] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. Battaglia. Learning to simulate complex physics with graph networks. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 8459–8468. PMLR, 2020. URL <https://proceedings.mlr.press/v119/sanchez-gonzalez20a.html>.
- [27] X. Lou, Y. Yang, and C. Choi. Learning object relations with graph neural networks for target-driven grasping in dense clutter. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 742–748. IEEE, 2022.
- [28] H. Jiang, B. Huang, R. Wu, Z. Li, S. Garg, H. Nayyeri, S. Wang, and Y. Li. RoboEXP: Action-conditioned scene graph via interactive exploration for robotic manipulation. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 3027–3052. PMLR, 2025. URL <https://proceedings.mlr.press/v270/jiang25c.html>.
- [29] K. Hauser. The minimum constraint removal problem with three robotics applications. *The International Journal of Robotics Research*, 33(1):5–17, 2014.
- [30] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Perez. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4:265–293, 2021.
- [31] S. Cheng, C. R. Garrett, A. Mandlekar, and D. Xu. NOD-TAMP: Generalizable long-horizon planning with neural object descriptors. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 1310–1339. PMLR, 2025. URL <https://proceedings.mlr.press/v270/cheng25a.html>.
- [32] B. Cieřlar, L. P. Kaelbling, T. Lozano-Pérez, and J. Mendez-Mendez. Learning long-horizon action dependencies in sampling-based bilevel planning. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4235–4252. PMLR, 2025. URL <https://proceedings.mlr.press/v270/cieslar25a.html>.
- [33] Universal Robots. UR5 technical specifications. https://www.universal-robots.com/media/50573/ur5_bz.pdf, 2026. Accessed 2026-05-24.
- [34] Intel. Intel realsense depth camera D435. <https://www.intelrealsense.com/depth-camera-d435/>, 2026. Accessed 2026-05-24.
- [35] Wheeltec. Wheeltec flexible gripper. <https://www.wheeltec.net/product/html/?159.html>, 2026. Accessed 2026-05-24.
- [36] Bluepoint Touch. Bluepoint touch six-axis force sensor. <https://link-touch.com/productinfo/6198456.html>, 2026. Accessed 2026-05-24.
- [37] IKEA. SAMLA box, clear, 301.029.74. <https://www.ikea.com/us/en/p/samla-box-clear-30102974/>, 2026. Accessed 2026-05-24.

Appendix Overview

This Appendix provides additional details not included in the main paper, covering the definitions of the intervention labels, dataset statistics, predictor equations and ablations, planner metrics and robustness checks, and closed-loop MuJoCo terminal analyses. The sections follow the pipeline from label construction to dependency prediction, planning, and physical rollouts.

A1 Dataset Construction and Dependency Labels

The intervention procedure creates the supervision used by the dependency predictor. For each target sample, we store the observed clutter scene, candidate target grasps, object nodes, and object-grasp edges. The key label is grasp-conditioned. An object o_i is a blocker for candidate grasp g_j only if removing o_i changes the feasibility or execution progress of that same grasp. This differs from ordinary visibility, proximity, or object-level obstruction labels because the dependency is tied to a concrete target action. Candidate target grasps are produced through the parallel-jaw and suction interfaces, restricted to the target region, ranked by proposal score, filtered to near top-down approaches within 30° , deduplicated with 8 mm and 10° thresholds, and capped at 40 per modality from at most 80 raw proposals. Both proposal interfaces supply the dense candidate pool. Isolated-target candidates at the current target pose augment current-clutter proposals, with a 75° approach limit; fully invisible targets receive no isolated augmentation. Each retained grasp stores its modality, pose, approach and lift directions, score, pregrasp offset, and modality-specific contact or sealing geometry.

A1.1 Split and Graph Scale

The dense benchmark contains 500 multi-layer scenes, split by scene into 350 training, 75 validation, and 75 test scenes. Table A1-A reports the number of scenes, target samples, candidate grasps, and object-grasp edges in each split. The splits contain 9,090, 1,950, and 1,945 target samples, respectively. The observed graphs contain 281,602 object nodes and 6,766,097 labeled edges. Prediction and stored-label planning use the same 75-scene test split.

Table A1-A: Dataset statistics.

Split	Scenes	Samples	Obj. nodes	Grasps	OG edges	Edges/sample
train	350	9,090	196,975	218,949	4,728,186	520.2
val	75	1,950	42,722	47,254	1,030,373	528.4
test	75	1,945	41,905	47,025	1,007,538	518.0
all	500	12,985	281,602	313,228	6,766,097	521.1

Figure A1-A visualizes representative initial target samples from the benchmark.

A1.2 Dependency Label Definitions

This section gives the full mathematical label definitions and audit statistics. Each edge stores four binary labels rather than a single mutually exclusive class: progress-any, sufficient-removal, approach-progress, and lift-progress; the all-zero case is the no-dependency state. The two typed progress labels describe the execution stage affected by o_i for the fixed grasp g_j ; progress-any is their union. Sufficient-removal is a stronger sufficiency attribute indicating that removing o_i alone makes g_j feasible. It is included because the planner needs to know which individual removals can directly recover a grasp, but it is not a separate visual blockage type in the figures. The figures instead show grasp-level approach and lift states obtained by aggregating typed dependencies across objects. Let $V(\mathcal{S}, t, g_j)$ be the staged validator, with full-scene result $r_j = V(\mathcal{S}, t, g_j)$ and single-removal result $r_{ij}^- = V(\mathcal{S} \setminus \{o_i\}, t, g_j)$. It returns a feasibility bit $f(r)$, terminal stage $c(r)$, and a numeric progress rank $\rho(r)$ over proposal validity, approach collision, closing or sealing, lift failure,

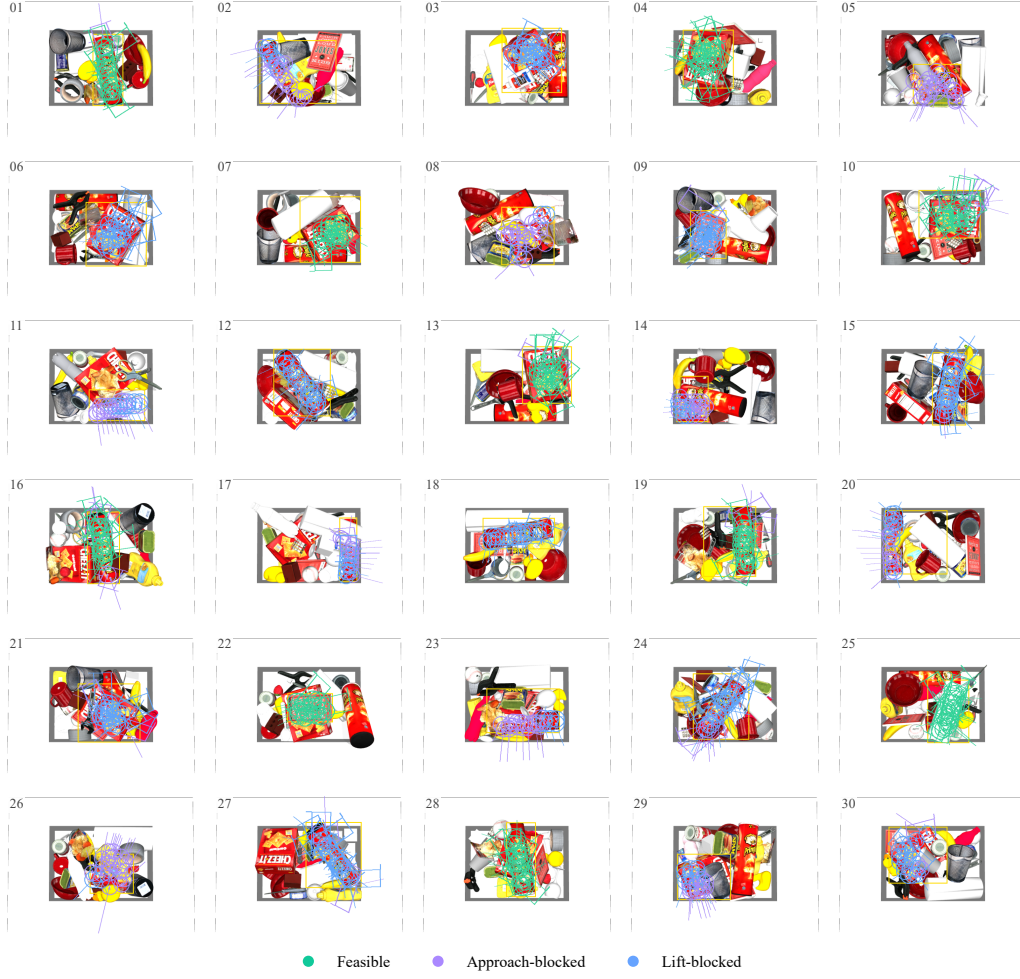


Figure A1-A: Representative initial target-grasp states in the benchmark. Each panel shows a cluttered target sample with the target object highlighted and the current parallel-jaw and suction candidates overlaid. The gallery contains 30 dense initial samples, with 10 examples each dominated by feasible, approach-blocked, or lift-blocked grasp outcomes; colors indicate the recorded grasp-level status. This illustrates that the benchmark contains both directly reachable targets and cases where candidate grasps fail for different physical reasons.

and success:

$$\rho(\text{invalid proposal}) = 0, \quad \rho(\text{approach collision}) = 1, \quad \rho(\text{closing/seal}) = 2, \\ \rho(\text{lift collision}) = \rho(\text{unstable lift}) = 3, \quad \rho(\text{success}) = 4.$$

The sufficient-removal label is

$$y_{ij}^{\text{succ}} = \mathbf{1}[\neg f(r_j) \wedge f(r_{ij}^-)].$$

Typed progress labels are

$$y_{ij}^{\text{app}} = \mathbf{1}[c(r_j) = \text{approach collision} \wedge \rho(r_{ij}^-) > \rho(r_j)], \\ y_{ij}^{\text{lift}} = \mathbf{1}[c(r_j) \in \{\text{lift collision, unstable lift}\} \wedge \rho(r_{ij}^-) > \rho(r_j)].$$

We set $y_{ij}^{\text{prog}} = y_{ij}^{\text{app}} \vee y_{ij}^{\text{lift}}$ and train on $\mathbf{y}_{ij} = [y_{ij}^{\text{prog}}, y_{ij}^{\text{succ}}, y_{ij}^{\text{app}}, y_{ij}^{\text{lift}}]$. Single-object tests are skipped and assigned zero dependency labels when the full-scene grasp is already feasible or when the terminal failure cannot be affected by removing a clutter object. These skipped cases include

bin-wall or support-surface collisions, invalid approach directions, and closing or sealing failures caused by the proposal direction or contact geometry. The distinction between sufficient-removal and typed progress remains useful for auditing. y_{ij}^{succ} is a final-success recovery label, while y_{ij}^{app} and y_{ij}^{lift} are stage-progress labels that can be positive even when more than one blocker must be removed. Table A1-B shows that all positive labels are sparse, which is why AP is the primary dependency-prediction metric.

Table A1-B: Dependency-label distribution.

Split	Progress-any	Sufficient	Approach	Lift
train	65,316 (1.38%)	31,645 (0.67%)	44,895 (0.95%)	20,421 (0.43%)
val	13,424 (1.30%)	6,515 (0.63%)	9,428 (0.92%)	3,996 (0.39%)
test	14,280 (1.42%)	7,029 (0.70%)	9,506 (0.94%)	4,774 (0.47%)
all	93,020 (1.37%)	45,189 (0.67%)	63,829 (0.94%)	29,191 (0.43%)

A1.3 Minimal Blocker Sets

The benchmark also stores an oracle minimal blocker set (MBS) size for each target sample when it can be solved within the bounded removal search. This quantity is used only for evaluating intervention cost and diagnostic headroom, not for training the predictor. For each fixed grasp, subsets are enumerated up to the oracle search budget $K_{\text{oracle}} = 3$, the maximum number of non-target objects allowed in the tested removal subset. If no subset succeeds, the grasp is treated as unrecoverable under the oracle budget. The bounded minimal set for grasp g_j is

$$\mathcal{B}_j^* = \arg \min_{\mathcal{B} \subseteq \mathcal{O}_t} |\mathcal{B}| \quad \text{s.t.} \quad |\mathcal{B}| \leq K_{\text{oracle}}, \quad V(\mathcal{S} \setminus \mathcal{B}, t, g_j). \text{feasible} = 1.$$

When a feasible set exists, the stored oracle removal order greedily selects the remaining object that produces the largest progress-rank improvement. Table A1-C reports 2,119 directly reachable targets and 6,830 targets requiring one to three removals. The remaining 2,002 targets are unsolved within the search depth and 2,034 have no proposals; percentages use the target count of the corresponding split. This distribution motivates reporting both target-retrieval success and removal count in the planning evaluation.

Table A1-C: Minimal blocker set sizes.

Split	$ \mathcal{B}^* = 0$	$ \mathcal{B}^* = 1$	$ \mathcal{B}^* = 2$	$ \mathcal{B}^* = 3$
train	1,494 (16.4%)	2,041 (22.5%)	1,642 (18.1%)	1,109 (12.2%)
val	330 (16.9%)	420 (21.5%)	359 (18.4%)	237 (12.2%)
test	295 (15.2%)	443 (22.8%)	324 (16.7%)	255 (13.1%)
all	2,119 (16.3%)	2,904 (22.4%)	2,325 (17.9%)	1,601 (12.3%)

A1.4 Grasp Modalities

The action representation supports parallel-jaw and suction grasps. Table A1-D reports 161,304 parallel-jaw and 151,924 suction candidates, including isolated-target completion. Each candidate retains its pose, score, approach and lift directions, and modality-specific jaw or sealing geometry.

Table A1-D: Grasp modalities.

Split	Parallel jaw	Suction	Total grasps
train	112,521 (51.4%)	106,428 (48.6%)	218,949
val	24,462 (51.8%)	22,792 (48.2%)	47,254
test	24,321 (51.7%)	22,704 (48.3%)	47,025
all	161,304 (51.5%)	151,924 (48.5%)	313,228

A2 Grasp-Conditioned Dependency Prediction

The predictor estimates, for every observed object o_i and candidate target grasp g_j , whether o_i blocks g_j . Prediction results report means and sample standard deviations over model seeds 7, 11, 23, with the fixed 350/75/75 scene split. The Hetero-GNN is trained with AdamW using learning rate 10^{-3} , weight decay 10^{-4} , batch size 32, and 45 epochs. No learning-rate scheduler, early stopping, or mixed-precision training is used; training runs for the full 45 epochs, and checkpoints are selected by validation AP for the progress-any label. The training workstation has an NVIDIA GeForce RTX 5090 Laptop GPU with 24 GB memory. The retained training protocol uses full-edge binary cross entropy with capped positive weights.

A2.1 Predictor Input and Output Definitions

The predictor takes a target identity, observed object regions with registered depth, and candidate target grasps. Simulation provides RGB-D and instance masks. On hardware, YOLO supplies RGB boxes and registered depth selects the corresponding target point cloud. Candidate centers projected inside the target region assign grasp proposals to that target. These observations are converted to structured graph tensors before prediction.

Table A2-A: Predictor inputs and outputs.

Tensor	Channels and construction
Object (36)	Box coordinates/confidence, visibility, depth statistics, estimated center/size, target-relative offsets, and identity encoding.
Grasp (28)	Modality/score, SE(3) pose, approach/lift/closing directions, contact normal and location, pregrasp offset, jaw width or suction radius.
Edge (22)	Relative position/depth, overlap, approach/lift segment distances and clearances, and normalized motion alignment.
Output (4)	Progress-any, sufficient-removal, approach-progress, and lift-progress probabilities for each object–grasp pair.

For an estimated object center $\hat{\mathbf{c}}_i$, motion segment L_j , and radius $\hat{r}_i$, path clearance is $d(\hat{\mathbf{c}}_i, L_j) - \hat{r}_i$. Alignment is the dot product between the normalized object-to-grasp vector and the approach or lift direction. The network consumes these structured features, not raw RGB-D or point clouds. Simulator contacts and intervention outcomes are used for supervision and evaluation rather than as predictor inputs.

A2.2 Edge-Context Model Details

Each scene-target sample is represented as a bipartite object-grasp relation

$$\mathcal{H}_t = (V_o, V_g, E_{og}), \quad E_{og} = V_o \times V_g.$$

Let $\mathbf{x}_i^o$, $\mathbf{x}_j^g$, and $\mathbf{e}_{ij}$ be the object, grasp, and object-grasp edge features. Object and grasp features are embedded independently,

$$\mathbf{h}_i^o = \phi_o(\mathbf{x}_i^o), \quad \mathbf{h}_j^g = \phi_g(\mathbf{x}_j^g),$$

and the edge-context branch initializes a dependency-edge state

$$\mathbf{u}_{ij}^0 = \phi_e(\mathbf{x}_i^o \oplus \mathbf{x}_j^g \oplus \mathbf{e}_{ij}),$$

where $\oplus$ denotes feature concatenation. For $\mathcal{N}_o(j) = \{i : (i, j) \in E_{og}\}$ and $\mathcal{N}_g(i) = \{j : (i, j) \in E_{og}\}$, the model computes per-grasp and per-object edge contexts at layer ℓ :

$$\bar{\mathbf{u}}_j^\ell = \frac{1}{|\mathcal{N}_o(j)|} \sum_{i \in \mathcal{N}_o(j)} \mathbf{u}_{ij}^\ell, \quad \bar{\mathbf{u}}_i^\ell = \frac{1}{|\mathcal{N}_g(i)|} \sum_{j \in \mathcal{N}_g(i)} \mathbf{u}_{ij}^\ell.$$

The edge state is then updated by residual context blocks:

$$\mathbf{u}_{ij}^{\ell+1} = \text{LN}[\mathbf{u}_{ij}^\ell + \psi_\ell(\mathbf{u}_{ij}^\ell \oplus \bar{\mathbf{u}}_j^\ell \oplus \bar{\mathbf{u}}_i^\ell)].$$

This aggregation makes each candidate dependency aware of other objects that may block the same grasp and of how the same object relates to alternative grasps, without relying on generic object-object or grasp-grasp propagation.

The decoder combines endpoint embeddings, object-grasp edge features, the final edge state, edge-context summaries, and raw observable features. One equivalent representation is

$$\mathbf{r}_{ij} = \mathbf{h}_i^o \oplus \mathbf{h}_j^g \oplus \mathbf{e}_{ij} \oplus \mathbf{u}_{ij}^L \oplus \bar{\mathbf{u}}_j^L \oplus \bar{\mathbf{u}}_i^L \oplus \mathbf{x}_i^o \oplus \mathbf{x}_j^g.$$

The classifier uses a shared trunk with label-specific heads.

$$\ell_{ij} = \delta(\mathbf{r}_{ij}), \quad \hat{\mathbf{p}}_{ij} = \sigma(\ell_{ij}) = [\hat{p}_{ij}^{\text{prog}}, \hat{p}_{ij}^{\text{succ}}, \hat{p}_{ij}^{\text{app}}, \hat{p}_{ij}^{\text{lift}}].$$

For the labeled edge set $\mathcal{E}_{\text{lab}}$, training minimizes weighted binary cross entropy over the four labels.

$$\mathcal{L}_{\text{dep}} = \frac{1}{|\mathcal{E}_{\text{lab}}| |\mathcal{K}|} \sum_{(i,j) \in \mathcal{E}_{\text{lab}}} \sum_{k \in \mathcal{K}} \lambda_k \ell_{\text{BCE}}^{\omega_k}(y_{ij}^k, \ell_{ij}^k) + \lambda_{\text{con}} \mathcal{L}_{\text{con}},$$

where $\mathcal{K} = \{\text{prog}, \text{succ}, \text{app}, \text{lift}\}$, λ_k are label weights, and ω_k are positive-class weights. The per-label weighted BCE is

$$\ell_{\text{BCE}}^{\omega}(y, \ell) = -\omega y \log \sigma(\ell) - (1 - y) \log(1 - \sigma(\ell)).$$

The consistency term follows the definition $y^{\text{prog}} = y^{\text{app}} \vee y^{\text{lift}}$.

$$\mathcal{L}_{\text{con}} = \frac{1}{|\mathcal{E}_{\text{lab}}|} \sum_{(i,j) \in \mathcal{E}_{\text{lab}}} (\hat{p}_{ij}^{\text{prog}} - \text{sg}[\max(\hat{p}_{ij}^{\text{app}}, \hat{p}_{ij}^{\text{lift}})])^2,$$

where $\text{sg}[\cdot]$ stops gradients through the typed heads. These equations are included here for reproducibility; the main paper summarizes the edge-context aggregation.

A2.3 Metrics and Baselines

Positive dependency edges account for 1.37% of observed graph edges overall and 1.42% in the test split, so the primary ranking metric is AP. Planner-facing metrics are computed at validation-selected thresholds and include F1, blocker set IoU, and oracle MBS recovery. The baseline ladder separates simple priors from learned edge and graph models. It includes random scores, center-distance scoring, a geometry heuristic, Edge-MLP, frozen G2N2-style graph scoring, and the final edge-context dependency model.

Table A2-B: Dependency prediction.

Method	AP $\uparrow$	AUC $\uparrow$	F1 $\uparrow$	Blocker IoU $\uparrow$
Random	1.4 $\pm$ 0.0	50.3 $\pm$ 0.4	2.8 $\pm$ 0.0	1.4 $\pm$ 0.0
Center-dist.	2.3 $\pm$ 0.0	65.4 $\pm$ 0.0	4.7 $\pm$ 0.0	2.9 $\pm$ 0.0
Geometry	3.3 $\pm$ 0.0	72.6 $\pm$ 0.0	7.1 $\pm$ 0.0	4.4 $\pm$ 0.0
Edge-MLP	31.5 $\pm$ 0.5	94.8 $\pm$ 0.2	37.3 $\pm$ 0.4	24.8 $\pm$ 0.3
G2N2	38.3 $\pm$ 0.5	95.7 $\pm$ 0.3	42.2 $\pm$ 0.5	29.3 $\pm$ 0.3
Ours	40.0 $\pm$ 0.3	96.4 $\pm$ 0.2	43.5 $\pm$ 1.1	30.7 $\pm$ 1.1

Table A2-B shows that simple score priors remain near the sparse-positive floor. Learned edge and graph models improve on simple geometric priors. The edge-context model has the highest AP, F1, and edge-derived blocker set IoU among these prediction baselines.

A2.4 Minimal Blocker-Set Recovery

The planner consumes ranked object lists for each candidate target grasp, so AP alone does not fully describe whether a predictor identifies the objects that belong to the oracle MBS. Table A2-C evaluates nontrivial target-grasp cases whose oracle MBS is nonempty. Recall@K measures whether objects in the oracle MBS appear in the top K predicted objects for the selected grasp, while blocker

Table A2-C: Minimal blocker set recovery.

Method	AP $\uparrow$	R@3 $\uparrow$	R@5 $\uparrow$	Blocker IoU $\uparrow$	Exact $\uparrow$
Object-only	12.3 $\pm$ 0.1	43.7 $\pm$ 0.9	62.6 $\pm$ 0.9	19.6 $\pm$ 0.7	8.7 $\pm$ 0.6
Edge-MLP	31.5 $\pm$ 0.5	47.8 $\pm$ 1.0	66.7 $\pm$ 1.1	23.5 $\pm$ 0.5	10.9 $\pm$ 0.2
G2N2	38.3 $\pm$ 0.5	48.9 $\pm$ 0.7	67.2 $\pm$ 0.7	25.6 $\pm$ 0.1	12.7 $\pm$ 0.4
Ours	40.0 $\pm$ 0.3	54.0 $\pm$ 0.7	73.3 $\pm$ 1.2	28.4 $\pm$ 1.7	14.6 $\pm$ 1.3

set IoU and exact match compare the predicted top set with the oracle MBS. The object-only baseline predicts blocker scores without distinguishing candidate target grasps; it is included to test whether the task can be reduced to target-level obstruction.

The edge-context model has the highest MBS recovery across recall and IoU metrics. The object-only baseline recovers many blockers in the top few objects because cluttered scenes often contain globally intrusive objects, but its much lower edge AP and lower blocker set IoU show that it cannot reliably assign blockers to the correct candidate grasp.

A2.5 Typed Dependency Labels

The main predictor is selected on progress-any AP, but the typed labels reveal where the task is difficult. Table A2-D reports per-label AP. Approach dependencies are easier because they are more directly tied to swept-path geometry. Lift dependencies are harder because the responsible object can be weakly visible or only become problematic after the grasp has already been established. These object-grasp labels should be distinguished from grasp-level blockage states used in visualizations. A grasp is approach-and-lift blocked when its incident edges include both typed dependency kinds, even if the approach and lift dependencies come from different objects.

Table A2-D: Per-label average precision.

Method	Progress-any	Sufficient	Approach	Lift
Random	1.43 $\pm$ 0.02	0.70 $\pm$ 0.01	0.95 $\pm$ 0.00	0.48 $\pm$ 0.00
Center-dist.	2.27 $\pm$ 0.00	0.81 $\pm$ 0.00	1.94 $\pm$ 0.00	0.48 $\pm$ 0.00
Geometry	3.25 $\pm$ 0.00	1.14 $\pm$ 0.00	2.85 $\pm$ 0.00	0.62 $\pm$ 0.00
Edge-MLP	31.51 $\pm$ 0.46	12.08 $\pm$ 0.84	37.07 $\pm$ 0.69	6.72 $\pm$ 0.69
G2N2	38.34 $\pm$ 0.49	16.96 $\pm$ 1.10	44.40 $\pm$ 0.43	9.73 $\pm$ 0.91
Ours	39.96 $\pm$ 0.27	22.06 $\pm$ 0.67	44.04 $\pm$ 0.90	15.82 $\pm$ 1.03

A2.6 Architecture Ablations

The final predictor is deliberately narrower than earlier heterogeneous message-passing variants. It keeps explicit object-grasp edge attributes and per-grasp/per-object edge-context aggregation, but removes attention, object-grasp cross-message passing, generic object-object/grasp-grasp propagation, and product/difference interaction terms. Table A2-E compares these architectural choices. Removing edge-context aggregation causes the largest performance drop, while adding generic OO/GG propagation lowers AP to 39.2. Pair interactions raise AP to 40.6 and F1 to 44.3, with blocker IoU of 31.4 compared with 30.7 for the retained model. Removing the raw decoder skip yields 40.1 AP.

A2.7 Error Slices

Lift-stage dependencies remain a difficult prediction slice, where the responsible object can become problematic after the target grasp has been established. This behavior is consistent with Table A2-D: approach dependencies have much higher AP than lift dependencies for every learned model. The error analysis therefore points to residual perception and lift-contact ambiguity in the object-grasp prediction task.

Table A2-E: Architecture ablations.

Variant	AP $\uparrow$	F1 $\uparrow$	Blocker IoU $\uparrow$
Edge-context model	40.0 $\pm$ 0.3	43.5 $\pm$ 1.1	30.7 $\pm$ 1.1
w/o edge-context aggregation	34.5 $\pm$ 0.7	39.0 $\pm$ 0.1	26.1 $\pm$ 0.3
w/o OG edge attributes	39.3 $\pm$ 0.8	42.9 $\pm$ 0.4	30.1 $\pm$ 0.5
w/o raw decoder skip	40.1 $\pm$ 0.9	43.5 $\pm$ 1.0	30.6 $\pm$ 1.0
+ pair interactions	40.6 $\pm$ 0.6	44.3 $\pm$ 0.1	31.4 $\pm$ 0.1
+ generic OO/GG	39.2 $\pm$ 0.6	43.3 $\pm$ 0.3	30.2 $\pm$ 0.3

A3 Minimal-Intervention Planning

The planning evaluation asks whether predicted object-grasp dependencies help retrieve a target while removing fewer objects. It uses the 1,945-target dense test cohort. Stored-label planning and single-target closed-loop evaluation use the validation-selected checkpoints from model seed 7. The protocol is a dataset-level deletion evaluation. After each planned removal, the object is deleted from the stored target sample and the planner re-evaluates the remaining objects and grasps using the saved intervention labels. Closed-loop MuJoCo rollouts are evaluated separately in Section A4. The comparison reports success and removals as coupled objectives rather than reducing the result to raw success.

A3.1 Metrics

Let $\mathcal{T}$ be the set of test target samples, a a planner, $m_a(x)$ the number of removals for sample x , and $z_a(x) \in \{0, 1\}$ the target retrieval success indicator after those removals. The main metrics are

$$\text{Success}(a) = \frac{1}{|\mathcal{T}|} \sum_{x \in \mathcal{T}} z_a(x), \quad \text{Removals}(a) = \frac{1}{|\mathcal{T}|} \sum_{x \in \mathcal{T}} m_a(x).$$

If $m^*(x)$ is the oracle MBS size for the grasp selected by planner a on sample x , the selected-grasp removal gap is

$$\text{Gap}_{\text{rem}}(a) = \frac{1}{|\mathcal{T}|} \sum_{x \in \mathcal{T}} (m_a(x) - m^*(x)).$$

This quantity is computed relative to the planner-selected grasp, not a globally optimal target-level MBS, so it can be negative when the planner removes fewer objects than that grasp’s stored oracle MBS size or when an unrecoverable case contributes only a diagnostic reference value. For removed set $\hat{\mathcal{B}}_a(x)$ and the selected grasp’s oracle MBS $\mathcal{B}^*(x)$, blocker IoU is

$$\text{IoU}(a) = \frac{1}{|\mathcal{T}|} \sum_{x \in \mathcal{T}} \frac{|\hat{\mathcal{B}}_a(x) \cap \mathcal{B}^*(x)|}{|\hat{\mathcal{B}}_a(x) \cup \mathcal{B}^*(x)|}.$$

When both sets are empty, the per-sample IoU term is defined as 1. The validation sweep uses

$$U(a) = \text{Success}(a) - \lambda_{\text{rem}} \text{Removals}(a), \quad \lambda_{\text{rem}} = 0.08,$$

only to select planner hyperparameters; the main reported test metrics remain success and removal count. This is the same utility score reported as “Utility” in the planning tables.

A3.2 Budgeted Dependency Planner

The main paper summarizes the planner at a high level. This section specifies the operating point used in the experiments. The planning budget K is the maximum number of non-target removals allowed during one retrieval episode. The main operating point uses $K = 5$, while the budget sweep reports $K \in \{1, 2, 3, 5\}$. Validation tuning selects the sufficient-removal dependency score for the main planner, with $\alpha = 0.5$, $\lambda_R = 1.0$, $\lambda_K = 0.002$, and $\lambda_D = 0.0$. In closed-loop mode, the planner removes only the first object in the selected prefix, updates the remaining object set, and

re-solves the same budgeted problem. It attempts the target grasp when the selected prefix is empty or terminates when the removal budget is exhausted.

The thresholded blocker set planner is evaluated as an ablation. It forms

$$\hat{\mathcal{B}}(g_j) = \{o_i \in \mathcal{O}_{\text{rem}} \mid D_{ij} > \tau\},$$

selects a grasp by thresholded blocker count, grasp score, and residual risk, and removes objects from the predicted set. The planner ablations report the effect of this thresholded interface under calibrated edge thresholds.

A3.3 Main Planning Comparison

Table A3-A compares the main planner baselines. Direct target grasping tests whether the target is already reachable. Random and nearest-to-target clearing test non-dependency object removal. Mechanical Search and X-Ray-style rows are target-centric prior-method adapters evaluated with the same deletion protocol. Highest-dependency and Top 5 dependency removal are aggressive learned-score baselines that tend to remove the full budget. The proposed predicted MBS planner uses the final dependency predictor and a validation-selected removal budget.

Table A3-A: Retrieval planning.

Method	Success $\uparrow$	Removals $\downarrow$	Utility $\uparrow$	Rem. gap $\downarrow$
Direct target grasp	8.3	0.00	0.083	-0.73
Random removal	13.3	5.00	-0.267	4.27
Nearest-to-target	26.2	5.00	-0.138	4.27
Mechanical Search	11.5	2.32	-0.071	1.59
X-Ray-style	14.9	2.32	-0.037	1.59
Highest dep.	40.5	5.00	0.005	4.27
Ours	31.9	2.32	0.134	1.73
Oracle MBS	67.7	0.95	0.601	0.00

Aggressive clearing can raise success by removing many objects, while the selected operating point balances retrieval success against intervention cost.

A3.4 Baseline Adapter Fairness

Mechanical Search and X-Ray-style methods were designed for target-centric retrieval and do not produce object-grasp dependency scores. To compare them in this benchmark without giving them privileged access to our labels, we implement them as target-centric removal adapters: Mechanical Search removes visible clutter according to a non-target priority rule, and the X-Ray-style adapter scores objects by target-support reduction cues rather than by learned object-grasp dependencies.

All stored-label planning methods are evaluated on the same 1,945 dense test targets, the same candidate target grasps, the same removal budget family, and the same deletion-and-replanning protocol. A method succeeds only if the selected target grasp becomes feasible after its chosen removals under the saved intervention labels. Prior-method adapters use observable target/object geometry and their adapter-specific ranking scores; they do not use oracle MBS annotations, ground-truth object-grasp dependency labels, or the learned dependency predictor.

The comparison separates three roles. Direct, random, nearest-to-target, Mechanical Search, and X-Ray-style rows are nonlearned or prior-method references. Highest-dependency, Top 5 dependency removal, and the no-removal-penalty ablation are aggressive learned-score references that show how much raw success can be purchased by spending the full budget. Oracle edge-label and Oracle MBS rows are diagnostic privileged-information references, not deployable baselines. The main operating point is selected by validation utility so that it is compared as a minimal-intervention policy rather than as a raw-success maximizer.

A3.5 Paired Bootstrap Comparisons

Figure A3-A shows paired bootstrap deltas between the default predicted MBS planner and key references. The paired protocol resamples the 75 scene clusters containing the same 1,945 dense targets, using 10,000 draws and target-weighted means, so each delta compares methods on identical clutter instances. Against nearest-to-target clearing, the default planner is slightly higher in success and much lower in removals. Against highest-dependency and Top 5 dependency removal clearing, it sacrifices raw success but improves utility by avoiding full-budget clearing. The Oracle MBS row uses privileged minimal blocker-set annotations and measures the remaining gap to stored set-level dependency information. It is a diagnostic reference under the bounded-search benchmark protocol, not an attainable learned or deployable baseline.

A3.6 Predictor-Source Ablation

Table A3-B holds the budgeted MBS planner fixed and changes only the source of dependency scores. This isolates the downstream value of the dependency predictor. The edge-context model gives the highest utility among learned grasp-conditioned predictors in this planner setting, ahead of Edge-MLP, G2N2-style scoring, and nonlearned heuristics. The object-only predictor reaches higher raw success while removing many more objects and producing lower blocker set IoU, so success and intervention cost are reported together. The Oracle edge-label row uses privileged edge-level dependency labels as planner scores, whereas the Oracle MBS rows use stored bounded minimal blocker-set annotations. Both are diagnostic privileged-information references rather than deployable learned baselines or unconstrained global upper bounds.

Table A3-B: Predictor ablations.

Predictor	Success $\uparrow$	Removals $\downarrow$	Rem. gap $\downarrow$	Blocker IoU $\uparrow$
Random	12.8	4.20	3.54	18.6
Center-dist.	11.2	4.20	3.78	18.6
Geometry	12.7	4.20	3.78	19.0
Edge-MLP	20.5	2.19	1.76	60.0
G2N2	25.1	2.27	1.75	58.9
Object-only	34.8	4.20	3.46	26.5
Ours	31.9	2.32	1.73	55.8
Oracle edge labels	21.4	0.13	-0.60	72.1

A3.7 Success-Biased Objective Selection

The default planner is selected for success-minus-removal utility. We also evaluate the fixed success-biased operating point and a matched object-only reference on the same test scenes. Figure A3-B summarizes the selected test operating points directly on the absolute metric scale.

The success-biased Ours setting reaches 39.0% success and 45.9% blocker IoU, compared with 32.4% and 28.6% for the object-only reference. It uses 3.24 rather than 3.28 removals, with utility 0.131 rather than 0.062. The default operating point keeps the lower-removal setting selected by validation utility.

A3.8 Planner Ablations

Table A3-C tests the planner components while keeping the final dependency predictor fixed. Concretely, the rows probe four parts of the planner: the thresholded blocker set interface, the validation-tuned budgeted objective, the set-selection heuristic itself, and the individual terms inside that objective. “Threshold blocker set planner” uses calibrated edge thresholds to define a predicted blocker set; “budgeted planner without validation tuning” keeps the same search family but uses untuned weights. “Naive edge-prefix MBS” simply takes the highest-ranked dependency edges in order, without the budgeted objective. “Progress-any only” and “sufficient-removal only” isolate the two

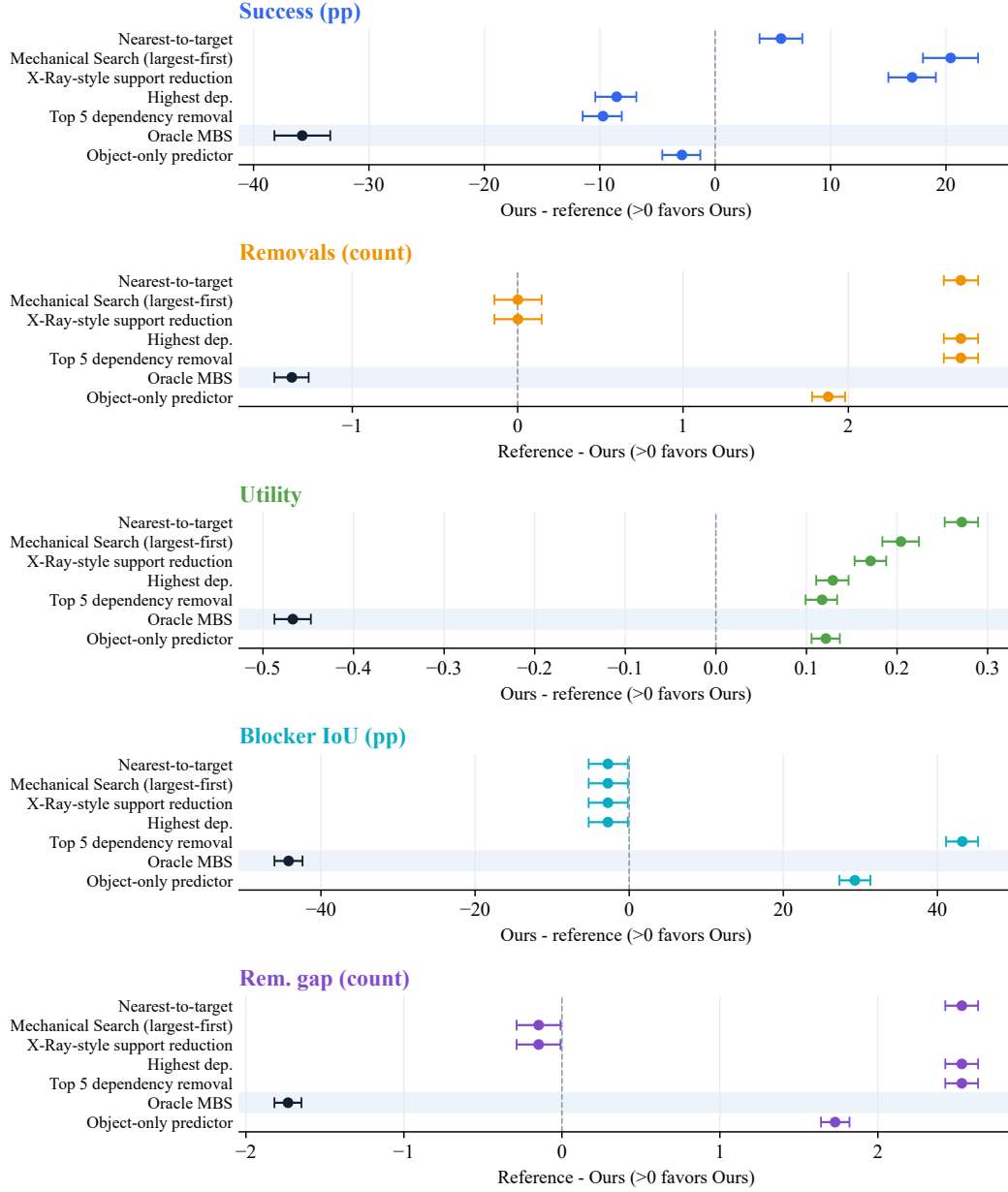


Figure A3-A: Paired differences for the default planner against key references on 75 test scenes (1,945 targets). Points show paired mean differences; bars show 95% scene-cluster bootstrap intervals from 10,000 resamples. The shaded row is the Oracle MBS reference; the dashed line is no difference.

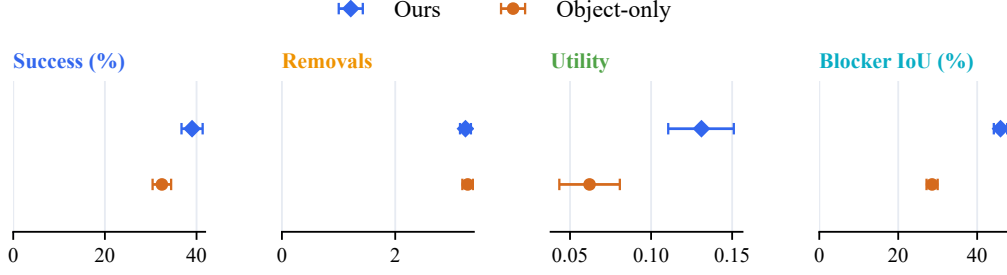


Figure A3-B: Success-biased Ours versus the matched object-only reference on 75 test scenes (1,945 targets). Points show absolute means; bars show 95% scene-cluster bootstrap intervals from 10,000 resamples.

dependency signals used to score a candidate removal set, while “budgeted open-loop plan” removes replanning after each deletion. The remaining rows ablate the reward terms inside the budgeted planner: the grasp-score bonus, the residual dependency risk term, and the removal-count penalty. “Top-k on highest-score grasp” is the aggressive clearing baseline that follows the highest-scoring grasp and removes its top-ranked blockers. The threshold and naive-prefix variants remove few objects and obtain low success. Removing the grasp-score term also lowers success, indicating that dependency scores must be paired with target-grasp quality. Removing the removal-count penalty raises raw success and increases removals. Open-loop planning remains close to closed-loop planning on this stored-label benchmark, and closed-loop replanning is used for the robot setting.

Table A3-C: Planner ablations.

Variant	Success $\uparrow$	Removals $\downarrow$	Utility $\uparrow$	Rem. gap $\downarrow$
Reference: tuned budgeted closed-loop	31.9	2.32	0.134	1.73
Threshold blocker set planner	3.4	0.01	0.033	-0.34
Budgeted planner without validation tuning	5.0	0.32	0.025	0.22
Naive edge-prefix MBS	1.7	0.04	0.014	-0.12
Budgeted progress-any only	27.1	2.30	0.087	1.80
Budgeted sufficient-removal only	31.9	2.32	0.134	1.73
Budgeted open-loop plan	30.9	2.29	0.125	1.73
No grasp-score bonus	5.1	0.47	0.014	0.36
No residual dependency cost	8.3	0.00	0.083	-0.73
No removal-count penalty	40.1	4.20	0.065	3.48
Top 5 dependency removal on highest-score grasp	41.6	5.00	0.016	4.27

A3.9 Prior-Method Adapter Sensitivity

Mechanical Search and X-Ray-style support-reduction baselines do not natively predict grasp-conditioned object-grasp dependencies, so they are evaluated through target-centric adapters. Table A3-D reports the adapter variants, including different X-Ray-style scoring terms and no-preemption settings. The X-Ray-style no-preemption row reaches higher success than its preempted counterpart by clearing the full budget, with more removals and lower validation-style utility than the proposed predicted MBS planner. The budget-3 and budget-5 rows for our method are included as operating-point references for budget-matched comparison. This sensitivity table is intended to make the adapter choice explicit: when a prior adapter is allowed to keep clearing without target preemption, it can improve raw success, but it does so by moving to the same high-removal regime as nearest-neighbor and highest-dependency clearing.

Table A3-D: Adapter sensitivity.

Variant	Success $\uparrow$	Removals $\downarrow$	Utility $\uparrow$	Rem. gap $\downarrow$
Mechanical Search random	9.7	2.32	-0.089	1.59
Mechanical Search largest-first	11.5	2.32	-0.071	1.59
Mechanical Search largest-first, no target preemption	18.6	5.00	-0.214	4.27
X-Ray-style full adapter	14.9	2.32	-0.037	1.59
X-Ray-style overlap only	14.6	2.32	-0.040	1.59
X-Ray-style distance only	13.7	2.32	-0.049	1.59
X-Ray-style frontness only	8.6	2.32	-0.099	1.59
X-Ray-style visible-area only	11.5	2.32	-0.071	1.59
X-Ray-style full adapter, no target preemption	30.2	5.00	-0.098	4.27
Ours budgeted, budget 3	21.9	1.35	0.111	0.85
Ours budgeted, budget 5	31.9	2.32	0.134	1.73

A4 Closed-Loop MuJoCo Evaluation

This Appendix section provides additional closed-loop MuJoCo and hardware experiments with summary plots, representative rollouts, and scene-sequence visualizations. The tables below retain the quantitative rollout summaries, while Figures A4-A and A4-B provide the corresponding visual summaries.

A4.1 Closed-Loop MuJoCo Rollout Analysis

Table A4-A reports terminal reasons for the 256-target dense rollout. The breakdown is kept here as the quantitative companion to the MuJoCo summary plots and sequence visualizations in Figures A4-A and A4-B.

Table A4-A: Single-target retrieval.

Method	N	Succ. (%)	Rem.	Approach	Lift	No prop.	Other
Direct	256	12.9	0.42	170	41	0	12
Nearest	256	20.3	4.81	116	45	0	43
Mechanical	256	36.7	3.61	98	38	0	26
X-Ray	256	48.4	3.36	61	39	0	32
Ours	256	51.2	2.53	46	33	0	46

In Table A4-A, N is the number of evaluated target-retrieval episodes. **Succ.** is the target-retrieval success rate, and **Rem.** is the mean number of non-target objects removed before termination. The terminal-reason columns count failed episodes: **Approach** denotes failures during the target-grasp approach, **Lift** denotes failures during post-grasp lifting or unstable lift, **No prop.** abbreviates no valid proposal available for the current target state, and **Other** collects any remaining termination cases. After shared proposal recovery, Direct attempts the target; Nearest removes the nearest non-target object; Mechanical and X-Ray are prior-method adapters; and Ours is the proposed dependency-guided closed-loop policy. The Other column includes unstable or out-of-bin resettling, exhausted recovery budgets, and step limits. All recovery removals count toward the five-removal budget.

The physical planner uses $\alpha = 0.75$, $\lambda_R = 0.2$, and $\lambda_K = 0.02$, plus a 0.02 coverage reward. It executes a selected prefix before the target attempt, allows up to two prefix phases, and regenerates proposals after physical updates. Mechanical, X-Ray, and Ours use target-feasibility checks over at most 12 candidates; Ours also allows validation-guided recovery. Each intervention is followed by up to 9,000 settling steps, and invalid resettling terminates the episode.

The ordered-sequence protocol uses four dense scenes, each containing 15 objects and ten designated targets. All five methods share the original target order and three additional permutations with order seeds 101, 211, and 307, giving 16 sequences and 160 targets per method on the same four

development scenes. Future targets that block the active target can be relocated within the bin and remain available for later retrieval. After an ordinary grasp failure the controller proceeds to the next designated target; a physically invalid state terminates the sequence, and unattempted targets remain in the completion denominator. Non-current objects are placed using the same geometry-checked candidate search and simulated settling preview for every method, with at most 32 placement previews. Accepted placements must remain within the bin and meet the settling criterion. All methods use multi-contact collision detection and up to 9,000 settling steps, with a 0.04 m/s velocity threshold. Table A4-B reports full-sequence success and partial progress. Ours retrieves 137/160 targets and completes 4/16 sequences with 183 relocations; X-Ray retrieves 129/160 and completes 3/16 with 281 relocations. R/retr. is total relocations divided by total completed targets. The Ours row uses model seed 11, previously selected from five model seeds by completion and then relocations on the original-order development runs; it is held fixed for the added orders. Across model seeds 7, 11, 23, 31, and 43, completion is 133, 137, 139, 134, and 128 out of 160, respectively, with 172, 183, 179, 163, and 161 relocations. The five-model mean is 83.9% completion and 171.6 relocations.

Table A4-B: Ordered retrieval.

Method	Seq. (%)	Targets	Comp. (%)	Reloc.	R/retr.	Terminal reasons
Direct	0	1.81/10	18.1	3.44	1.90	with failures 16
Nearest	6	5.25/10	52.5	50.00	9.52	with failures 15; complete 1
Mechanical	0	5.44/10	54.4	27.06	4.98	with failures 16
X-Ray	19	8.06/10	80.6	17.56	2.18	with failures 12; unstable 1; complete 3
Ours	25	8.56/10	85.6	11.44	1.34	with failures 12; complete 4

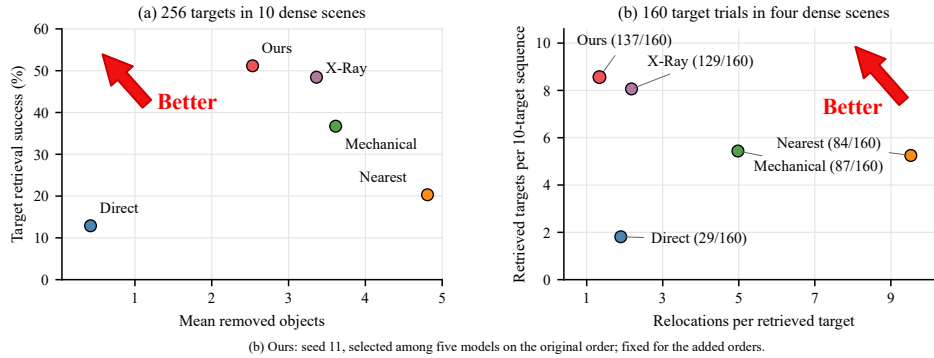


Figure A4-A: Closed-loop MuJoCo evaluation under two test settings. Left: the 256-target dense retrieval test plots target retrieval success against mean non-target removals. Right: the ordered protocol plots mean retrieved targets per ten-target sequence against relocations per retrieved target, using all 160 targets per method. The arrows indicate higher completion with fewer interventions. Ours uses the previously selected model seed 11, held fixed across the additional orders.

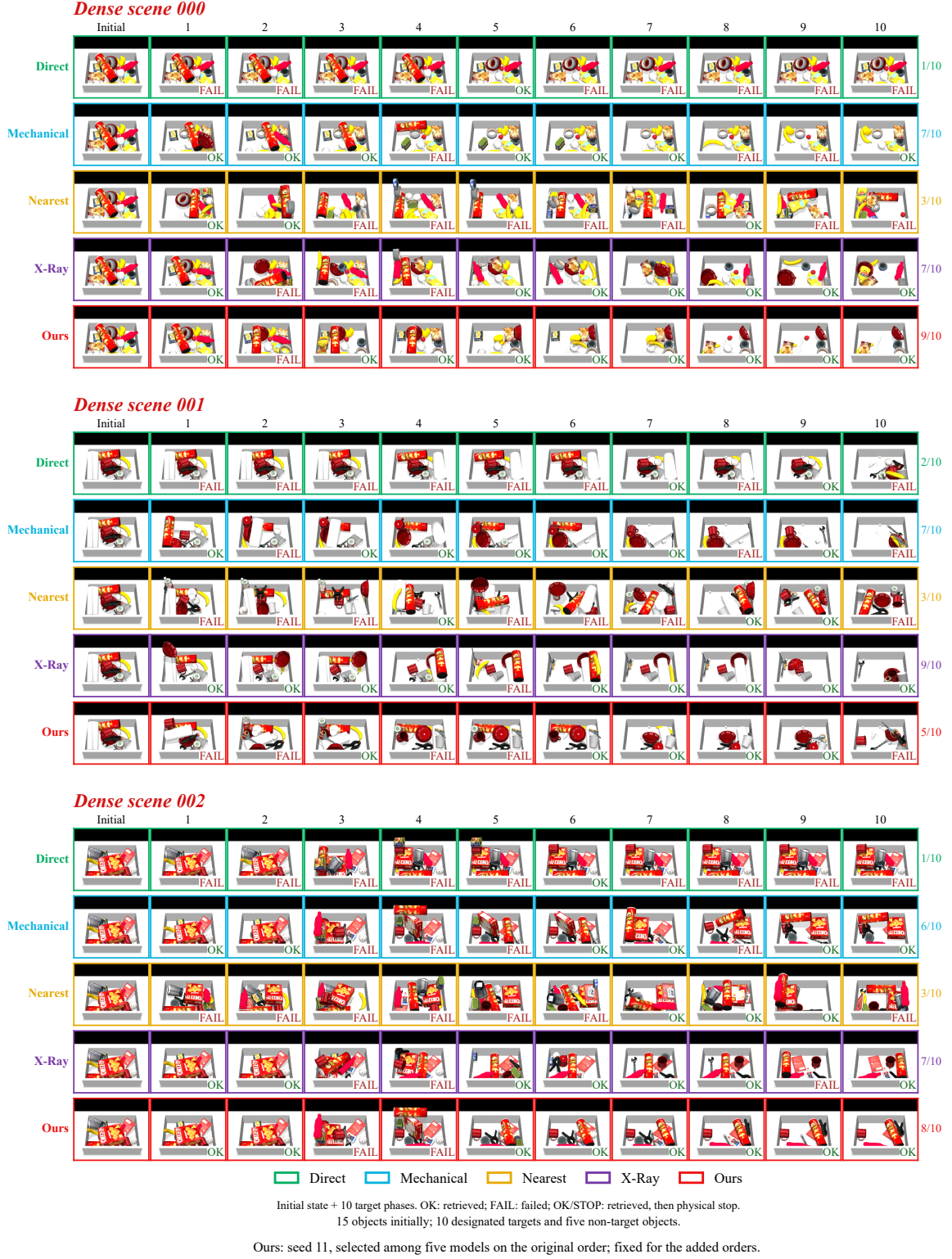


Figure A4-B: Closed-loop MuJoCo ordered-sequence visualization. The three scene groups show the first three scene roots under order seed 101, fixed before the additional baseline evaluations. Rows compare Direct, Mechanical, Nearest, X-Ray, and Ours; columns show the initial state and the state after each designated target phase. Colored borders identify methods, and the phase labels show retrieval outcomes. Ours uses model seed 11, previously selected on the original-order development runs.

A4.2 Real-Robot Platform and System Logic

The real-robot platform uses a UR5 manipulator [33], with both the eye-in-hand and eye-to-hand cameras implemented by Intel RealSense D435 sensors [34]. The end effector is a Wheeltec flexible gripper [35], and the wrist force sensing is provided by a Bluepoint Touch six-axis force sensor [36]. The WorkSpace uses an IKEA SAMLA clear box (Article Number 301.029.74) [37]. The adapter, suction cup, and magnet are custom-made. Figure A4-C summarizes the software and hardware logic of the platform, while Figure A4-D shows the physical real-robot system used for hardware validation.

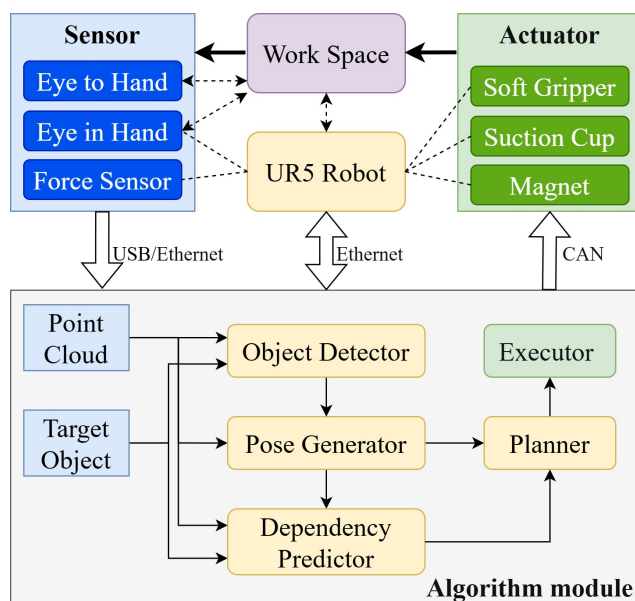


Figure A4-C: Real-robot hardware and software stack. Camera feed the object detector, pose generator, and dependency predictor with pointcloud via USB; the planner selects the next action; and the executor sends commands through the corresponding Ethernet and CAN links.

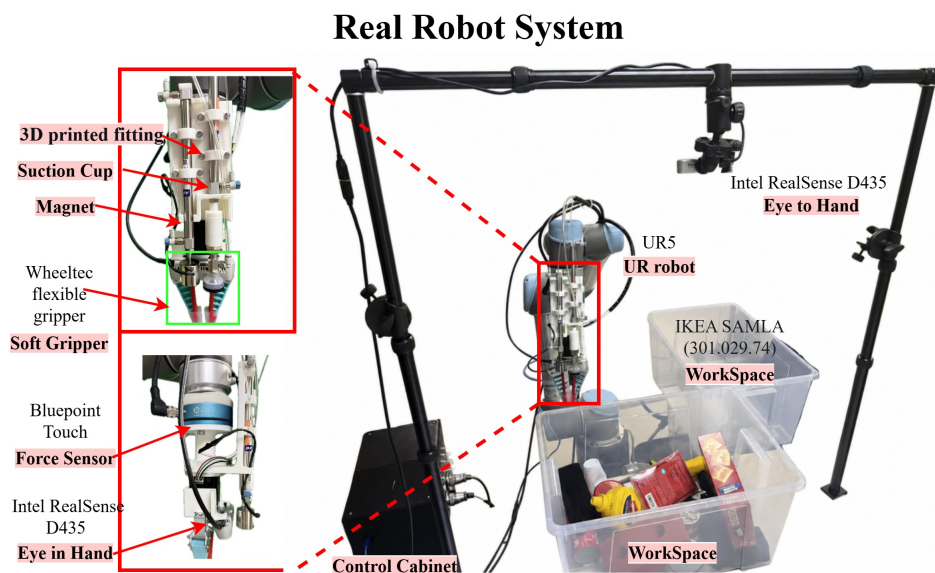


Figure A4-D: Real-robot system used for hardware validation.

Table A4-C: Repeated real-robot runs.

Method	Trials	Success runs	Removals runs	Mean success / removals
Direct	20	6 / 7 / 9	0 / 0 / 0	$7.3 \pm 1.5 / 0.0 \pm 0.0$
Mechanical	20	15 / 14 / 16	7 / 7 / 8	$15.0 \pm 1.0 / 7.3 \pm 0.6$
Ours	20	18 / 18 / 19	4 / 5 / 5	$18.3 \pm 0.6 / 4.7 \pm 0.6$

A4.3 Real-Robot Repeated Trials and Execution Example

The main paper reports the mean over three independent 20-trial hardware repetitions. Table A4-C lists the per-run success and removal counts used to compute those aggregates, so the variance across repeated hardware trials can be inspected directly. The hardware comparison includes Direct retrieval as a no-intervention reference, a Mechanical Search adapter as the nonlearned clearing baseline, and our dependency-guided policy. Direct never removes non-target objects, while the two clearing policies can move clutter before attempting the target; across the three repetitions, the proposed policy achieves higher target success with fewer removals than the hardware clearing baseline.

Figure A4-E shows an additional real-robot execution example on a randomly arranged clutter scene using our dependency-guided policy. The robot first observes the target and surrounding clutter, predicts the object-grasp dependencies for the available target-grasp candidates, and removes the selected non-target blockers before executing the target retrieval. This example illustrates that the learned policy can produce a compact clearing sequence in an unstructured hardware scene instead of exhaustively removing nearby objects. Additional execution and clearing processes are shown in the uploaded video.

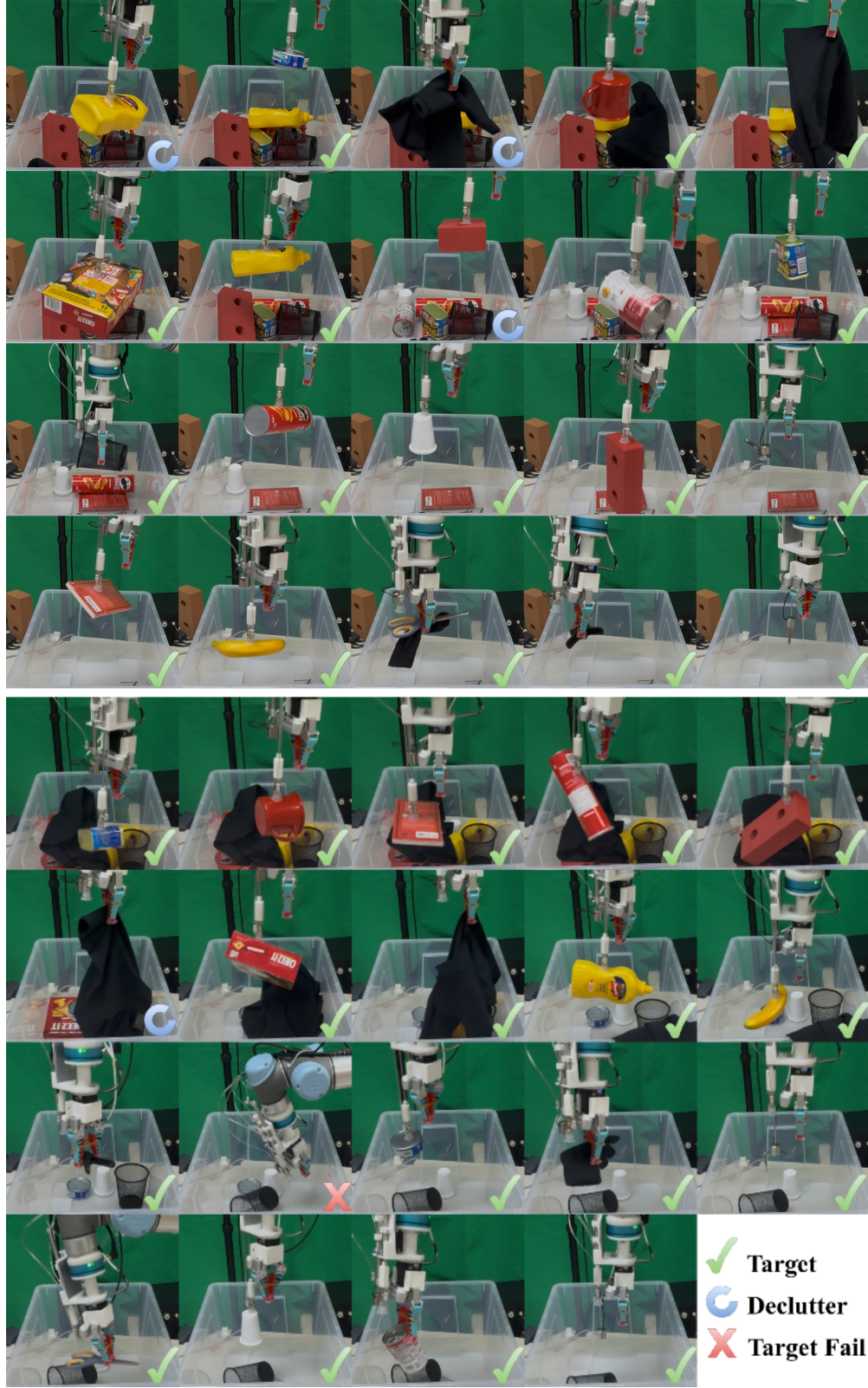


Figure A4-E: Additional real-robot execution example in a randomly arranged clutter scene using our method. The sequence shows the dependency-guided policy selecting and clearing predicted blockers before retrieving the target object.